

A continuación se proporciona una descripción detallada del proyecto en términos de inteligencia artificial (IA) centrado en un chatbot y un modelo de recomendaciones. El proyecto combina estratégicamente datos scrapeados y datos simulados para construir un conjunto de datos completo y enriquecido que alimenta tanto al chatbot como al modelo de recomendaciones.

Origen y Naturaleza de los Datos

El conjunto de datos utilizado en este proyecto es el resultado de una combinación estratégica entre datos scrapeados y datos simulados. Esta fusión se llevó a cabo con el propósito de construir un dataset completo y enriquecido, capaz de abastecer tanto al chatbot como al modelo de recomendaciones, asegurando así la diversidad y calidad de la información necesaria.

Datos Scrapeados:

Se obtuvieron mediante técnicas de web scraping de los siguientes sitios:

https://docs.google.com/document/d/1Mz1CPijtU_4R3-OT8zgNj75-ZJ-CDbmf/edit?usp=sharing&oid=116009425846680638457&rtpof=true&sd=true

Datos Simulados:

Adicionalmente, se incorporaron datos ficticios simulados para completar las posibles brechas en la información y para proporcionar un conjunto de entrenamiento más robusto al modelo. Estos datos simulados se generaron siguiendo patrones realistas y escenarios que podrían surgir en el contexto del sistema de recomendación y el chatbot.

Proceso de Limpieza y Preprocesamiento de Datos

Los datos recolectados fueron sometidos a un proceso exhaustivo de limpieza y preprocesamiento para garantizar la calidad y coherencia de la información. Se llevaron a cabo las siguientes actividades:

Limpieza de Datos

Manejo de Datos Faltantes: Se identificaron y abordaron los valores nulos o faltantes de manera adecuada, ya sea mediante imputación o eliminación según el contexto.

Detección y Tratamiento de Outliers: Se realizaron análisis estadísticos para identificar posibles valores atípicos que pudieran afectar la integridad de los resultados, y se aplicaron estrategias para gestionarlos.

Preprocesamiento.

Normalización y Escalado: Las variables numéricas fueron normalizadas o escaladas según la necesidad del modelo, para garantizar una comparación adecuada entre diferentes magnitudes.

Transformación de Variables Categóricas: Se realizaron conversiones y codificaciones de variables categóricas para integrarlas efectivamente en el modelo.

Elección del Algoritmo de Machine Learning

En esta etapa, se ha optado por utilizar el método de Diferenciación de Cosenos debido a su eficacia en problemas de filtrado colaborativo. El proceso de implementación y entrenamiento se ha llevado a cabo siguiendo los pasos detallados a continuación:

Elección del Algoritmo:

Se ha seleccionado el método de Diferenciación de Cosenos como base para el modelo de recomendación. La elección se fundamenta en su capacidad para medir la similitud entre vectores en un espacio multidimensional, lo que resulta crucial para proporcionar recomendaciones personalizadas a los usuarios.

Instanciación del Modelo:

Se ha creado una instancia del modelo utilizando una implementación adecuada proporcionada por la biblioteca correspondiente. Esto asegura una integración eficiente del algoritmo en el sistema de recomendación.

Entrenamiento:

El modelo se ha entrenado utilizando un conjunto de datos específico, donde el algoritmo ajusta sus parámetros internos para comprender las preferencias de los usuarios y las características de los productos. Este proceso es esencial para establecer la capacidad predictiva del modelo y garantizar recomendaciones precisas.

Componentes Principales del Algoritmo

El método de Diferenciación de Cosenos se compone de varios elementos clave que han sido utilizados en el proceso de implementación:

Medida de Similitud:

La diferenciación de cosenos proporciona una medida de similitud entre vectores en un espacio multidimensional. Esta medida se calcula como el coseno del ángulo entre dos vectores, lo que permite determinar la similitud semántica entre usuarios y productos en el contexto del sistema de recomendación.

Aplicación en Recomendación y Filtrado Colaborativo:

La aplicación de la diferenciación de cosenos en el filtrado colaborativo permite modelar la interacción entre usuarios y elementos. Al calcular la similitud coseno entre vectores de usuarios y vectores de productos, el algoritmo puede predecir las preferencias de los usuarios basándose en usuarios similares, mejorando así la precisión de las recomendaciones.

Regularización y Control de Overfitting:

Para garantizar la calidad y generalización del modelo, se ha utilizado la Regularización y Control de Overfitting. La introducción de términos de regularización ha ayudado a evitar el sobreajuste a los datos de entrenamiento, mejorando la capacidad del modelo para generalizar a nuevos datos y asegurando recomendaciones precisas y relevantes a largo plazo.

BASE DE DATOS**Modelo de Datos:**

En el caso de MongoDB y Parse Server, se utiliza un modelo de datos basado en documentos JSON. MongoDB es una base de datos NoSQL que almacena datos en documentos BSON (Binary JSON), y Parse Server utiliza MongoDB como su almacenamiento de datos predeterminado.

En este modelo, la información se organiza en documentos que son similares a objetos JSON, y estos documentos se almacenan en colecciones en lugar de tablas. Cada documento puede tener una estructura flexible, lo que significa que diferentes documentos en una misma colección pueden tener campos distintos.

1. Diagrama de Datos y Esquema de la base de datos:

<https://drive.google.com/drive/folders/1009pITJLCK1f02mCzTJEeM4CVPJucq0I?usp=sharing>

2. Datos clave-valor y Optimización Redis:

Caching de Consultas Frecuentes:

Parse Server identifica consultas o resultados de consultas que son frecuentemente utilizados pero no cambian con frecuencia.

Estos resultados se almacenan en Redis como pares clave-valor, donde la clave puede ser un identificador único asociado a la consulta y el valor es el resultado de la consulta.

Configuración de Parse Server:

Parse Server se configura para utilizar Redis como su sistema de almacenamiento en caché. Esto se hace mediante la configuración del parámetro `cacheAdapter` durante la inicialización.

Uso en Consultas:

Antes de realizar consultas a la base de datos principal, Parse Server verifica si los datos requeridos están presentes en la caché de Redis.

Si los datos están en la caché, se evita la consulta a la base de datos principal, mejorando significativamente el rendimiento.

Manejo de Expiración:

Parse Server configura tiempos de expiración para los datos almacenados en caché en Redis.

Esto asegura que los datos no estén obsoletos y se vuelvan a consultar de la base de datos principal periódicamente.

Reducción de Carga en la Base de Datos Principal:

Al almacenar en caché datos frecuentemente utilizados, Parse Server reduce la carga en la base de datos principal (por ejemplo, MongoDB).

Esto mejora el rendimiento general de la aplicación al disminuir la necesidad de realizar consultas intensivas en la base de datos.

Al implementar este enfoque de almacenamiento en caché con Redis, Parse Server logra una optimización significativa en términos de rendimiento al reducir la carga en la base de datos principal y al acelerar el acceso a datos frecuentemente utilizados.

3. Políticas de Seguridad:

En Parse Server con MongoDB, las políticas de seguridad están relacionadas con cómo se controla el acceso a la base de datos MongoDB y cómo se aseguran los datos almacenados. políticas de seguridad en este entorno:

Autenticación en MongoDB:

MongoDB utiliza un sistema de autenticación basado en roles y usuarios. Parse Server debe autenticarse correctamente con MongoDB para acceder a los datos.

Las credenciales de acceso (nombre de usuario y contraseña) deben ser seguras y únicas para garantizar la autenticación segura entre Parse Server y MongoDB.

Autorización y Roles:

MongoDB permite la asignación de roles y permisos a los usuarios. Parse Server puede definir roles específicos para acceder a ciertos recursos o colecciones en MongoDB.

Es posible restringir las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) basadas en roles para garantizar que los usuarios tengan solo los permisos necesarios.

Configuración de Parse Server:

La conexión entre Parse Server y MongoDB se establece de manera segura, utilizando conexiones cifradas (SSL/TLS) para proteger la transferencia de datos, en producción.

Políticas de Seguridad de Parse Server:

Parse Server implementa políticas de seguridad a nivel de aplicación, permitiendo definir quién tiene acceso a los datos y qué operaciones pueden realizar.

Se pueden configurar ACLs (Listas de Control de Acceso) en los objetos Parse para definir quién puede acceder, leer, escribir o eliminar esos objetos.